

SCL — Conception : intégrer l'action dans l'orchestrateur

Etienne Lamy

2026-07-21

Table des matières

0. Thèse	1
1. Le problème	2
2. Ce qui existe déjà (inventaire — ne pas réinventer)	2
3. Principe directeur : l'action, un opérateur typé de plus	3
4. La mécanique d'objectifs (le moteur)	4
5. Le déroulé continu (« le LLM parle en continu »)	5
6. Planification A^* à $g()/h()$ émergés	5
7. Renforcement nocturne « de plus en plus amont »	6
8. Exploiter tout ce qu'un nouveau module apporte	6
9. Alternatives considérées	7
10. Plan d'implémentation (étapes 16→20, chacune mesurable)	7
11. Risques & garde-fous	8

Comment l'action entre dans l'orchestrateur-LLM : objectifs, déroulé continu, planification A^ à $g()/h()$ émergés, renforcement nocturne de plus en plus amont.* Document de conception (précède l'implémentation), versionné `.md/.tex/.pdf`. 2026-07-21. Auteur : Etienne Lamy. Fait suite à `SCL_bilan_modules_orchestrateur.md`.

0. Thèse

Jusqu'ici l'orchestrateur composait des programmes de **perception** (compresser, générer, prédire un champ) vers une cible ancrée. **L'action est le même problème, avec un opérateur de plus : `agir(a)` fait avancer le monde d'un pas et rend le futur *prévisible* — pourvu qu'un module ait appris la conséquence de `a`.** La difficulté propre à l'action n'est pas la prédiction (elle émerge comme le reste)

mais l'**explosion combinatoire** de l'arbre des actions (comme aux échecs). On la maîtrise par trois leviers, tous déjà amorcés dans le code :

1. une **mécanique d'objectifs** qui, à chaque instant, désigne *ce vers quoi* chercher (donc élague brutalement l'arbre) ;
2. une **planification A*** dont `g()` (coût déjà payé) et `h()` (reste à faire) sont **des sorties de modules émergés**, évaluée **au plus haut niveau d'abstraction disponible** (chercher sur des états compacts, pas sur le champ brut) ;
3. un **renforcement nocturne** qui propage le crédit **de plus en plus en amont** : on mange d'abord un sucre par hasard, la nuit apprend à s'en approcher à 1 pas, puis 2, ... jusqu'à 6-15 pas — l'horizon dont ce monde a besoin pour naviguer.

Le tout tient dans un **déroulé continu** : l'orchestrateur « parle » sans cesse (il émet des actions et des prédictions de pensée), prend les entrées réelles au moins une fois, puis toutes les X étapes *imaginées* pousse l'action retenue, passe à T+1 et réinjecte les vraies entrées. C'est le design proposé par l'auteur ; ce document le formalise et le raccorde à l'existant.

1. Le problème

- **L'arbre d'actions est exponentiel.** 5 accélérations, un horizon de navigation de 6-15 pas → 5 à 5¹⁵ feuilles. Impossible à dérouler en entier ; il faut **chercher avec une direction** (objectif) et **une heuristique** (h).
- **Prévoir en champ brut est cher.** Dérouler 10 pas de champ 10×10 sous chaque action coûte 10×5^k passes conv. Prévoir sur un **état compact** (p. ex. « vecteur vers le sucre le plus proche », ou le latent d'un module de régime) est bien moins cher et **suffit à décider**. Il faut *encourager* la recherche à se placer haut dans le graphe de modules.
- **La récompense est rare et tardive.** Manger un sucre est un événement ponctuel ; sa cause utile (« j'ai accéléré vers lui il y a 8 pas ») est loin en amont. Sans propagation de crédit, l'agent ne relie jamais l'action lointaine à la récompense.

2. Ce qui existe déjà (inventaire — ne pas réinventer)

L'intégration est surtout un **assemblage**. Briques présentes et réutilisables :

Besoin de la conception	Brique existante	État
Espace d'action, faim/douleur	<code>monde.py</code> (5 accél., sucre/bâton, collisions)	prouvé
A* + heuristique apprise <code>h()</code>	<code>recherche.a_etoile</code> , <code>ValeurApprise</code> , <code>entraîner_v_psi</code> (TD)	prouvé (générique)
A* ancrée (g/h via points de vérité, confiance profondeur)	<code>recherche.a_etoile_ancree</code>	présent
Curiosité / progrès / maîtrise / frontière	<code>curiosite.py</code>	prouvé

Besoin de la conception	Brique existante	État
Besoin dominant (argmax+hystérésis), réflexe douleur	<code>decision_action.py</code>	prouvé
Prédicteurs action-conditionnés ($v \rightarrow v'$), naissance sur surprise	<code>dynamique.py</code>	prouvé
Rejeu contrefactuel nocturne, regret de composition	<code>credit.py</code>	présent
Orchestrateur-LLM à triplets, déroulé continu , action typée, REINFORCE	<code>attention.py</code> (Set Transformer + Pointer Net, <code>trace_autoreferentielle</code> , <code>macro_pas</code>)	présent
Émetteur de programmes typé conditionné par objectif (Mode B)	<code>mode_b.py</code>	prouvé (étapes 13-14)
Recherche typée par valeur $G - \cdot$ coût (Mode A)	<code>orchestrateur.py</code>	prouvé (étape 11)
Prédicteur de régime $\text{champ} \rightarrow \text{champ}$ (dynamique du monde)	<code>regime.py</code>	prouvé (étape 10)

Deux orchestrateurs coexistent : le **cœur-étapes** (`orchestrateur.py/mode_b.py`, testé, minimal, sans action) et l'**orchestrateur-LLM v6** (`attention.py/boucle.py`, riche, avec action/ A^* /curiosité, moins couvert par les tests). La conception **étend le cœur-étapes** (discipline de test, une étape mesurable à la fois) en **récoltant les algorithmes** de la v6 (A^* , curiosité, crédit, besoins). La `v6 attention.py` reste la cible d'un substrat LLM complet qui pourra, plus tard, remplacer l'émetteur GRU de Mode B sans changer les interfaces.

3. Principe directeur : l'action, un opérateur typé de plus

Le langage typé de l'orchestrateur (étape 11) a des types `champ`, `latent` et les opérateurs `compresser/generer/predire_champ/predire_latent`. On ajoute :

- un type **etat** (état compact du monde utile à la décision : latent de régime, ou vecteur d'objectif appris — voir §6) et un type **action** (élément de) ;
- l'opérateur **agir** : $(\text{etat}, \text{action}) \rightarrow \text{etat}$ — le **modèle de transition action-conditionné**. C'est LE nouveau module : il *rend le futur interprétable*, comme quand « décider de prendre un verre » n'énumère pas les positions des doigts mais s'appuie sur un générateur qui sait à quoi ressemble « après ».
- l'opérateur **predire_champ_sous_action** : $(\text{champ}, \text{action}) \rightarrow \text{champ}$ — la version bas-niveau (rollout en champ brut), coûteuse, gardée comme **point d'ancrage vérifiable** (§7.4 : on peut toujours redescendre au champ pour vérifier une prédiction d'état).

Un module **agir** naît **exactement** comme les autres : sur **surprise confirmée** (agir révèle une conséquence qu'aucun module ne prédit) + **grâce**, puis il est entraîné jusqu'à la maîtrise et **verrouillé** (§1.4). `dynamique.py` fait déjà cela pour $v \rightarrow v'$; on généralise à $\text{etat} \rightarrow \text{etat}$.

Conséquence structurante : dès qu'un module **agir** existe, l'orchestrateur dispose, sur son état compact, d'une **fonction de transition** — donc d'un **arbre de prévisions** qu'il peut dérouler *sans toucher au monde*. C'est là que vit la planification.

4. La mécanique d'objectifs (le moteur)

Un **vecteur de pulsions**, chacune un scalaire mesurable, sans mélange continu entre elles (§15.3 : un seul besoin *dominant* gouverne l'action, par `argmax` + hystérésis — `decision_action.priorisation_besoin_dominant`

pulsion	signal (mesuré)	rôle
faim	monte avec le temps, chute sur sucre (monde)	pulsion de corps ; oriente vers le sucre
douleur	pic sur bâton ; réflexe câblé non atrophié (<code>reflexe_cable</code>)	garde-fou, court-circuite tout
curiosité	incertitude prédictive haute (<code>curiosite.incertitude</code>)	va vers les zones mal prédites (découvrir)
apprentissage	progrès d'apprentissage > 0 (<code>progres_apprentissage</code>)	« jouer » : progresser sur une tâche déjà entamée
bullage	un module maîtrisé couvre la situation (<code>maitrise</code>)	agir en pilote automatique → libérer du calcul pour la prévision long terme
temps perdu	pas sans gain (ni pulsion réduite, ni progrès)	récompense négative faible à long horizon — anti-tergiversation

Points de conception :

- **Sélection discrète, pas de pot-commun.** Le besoin dominant `k_t` sélectionne l'**objectif** qui *conditionne* l'émetteur (Mode B émet déjà conditionné par l'objectif : on élargit le jeu d'objectifs de {prédire, reconstruire} à {réduire faim, éviter douleur, réduire incertitude, progresser, consolider}). L'hystérésis évite le papillonnage (déjà éprouvé sur les épisodes, étape 12).
- **Bullage = profondeur, pas inaction.** Agir sous un module *maîtrisé* coûte ~0 en calcul (prédiction fiable, peu de branches à explorer). Le **budget de calcul** ainsi libéré finance un **horizon de prévision plus long** (ou une consolidation type-nuit). C'est littéralement « faire sans effort ce qu'on maîtrise pour penser à autre chose ».
- **Temps perdu** = un terme de récompense `r_temps` < 0 par pas, **dominé** par tout vrai gain, mais **décisif quand rien d'autre ne bouge** : il pousse à agir plutôt qu'à tergiverser, à un horizon assez long pour ne pas être myope.

La récompense qui alimente la planification et le renforcement est donc : $r_t = \Delta \text{faim} + \Delta \text{douleur} + r_{\text{progrès}} + r_{\text{temps}}$, chaque terme mesuré, aucun câblé à la géométrie du monde (aucune coordonnée d'objet dans la voie de décision — dette §27 tenue).

5. Le déroulé continu (« le LLM parle en continu »)

Design retenu (celui de l’auteur, formalisé) — une boucle de type *model-predictive control* :

état réel $s_t \leftarrow$ perception du monde (au moins une fois)

répéter:

```
# PENSÉE (imaginée, sans toucher au monde) : l'orchestrateur "parle"
pour X étapes calculées:
    l'orchestrateur émet un pas : soit un triplet de perception (interpréter s),
    soit une action imaginée  $a \rightarrow s' = \text{agir}(s, a)$  # déroulé de l'arbre
    (plusieurs pistes explorées PUIS abandonnées - cf. §6)
# ACTE : on pousse UNE action réelle (la racine du meilleur plan)
 $a^* =$  première action du meilleur plan trouvé
monde  $\leftarrow$  appliquer_action( $a^*$ ) ;  $s_{\{t+1\}} \leftarrow$  perception réelle
pousser  $s_{\{t+1\}}$  dans le contexte ;  $t \leftarrow t+1$ 
```

- **Un seul flux, continu.** Perception et action sont des jetons du même déroulé (**attention.macro_pas** + **trace_autoreferentielle** réinjectent la sortie passée : le mécanisme existe). « Toutes les X étapes calculées, on passe à $T+1$ » = X pas *imaginés* entre deux pas *réels*.
- **Fenêtre temporelle** = l’horizon imaginé X (démarré à $\sim T+5$, l’horizon *naturel* mesuré étape 8, et **s’étend** quand le bullage libère du calcul). **Fenêtre dimensionnelle** = le nombre d’éléments pointables dans T_t (modules actifs + [NEW] + trace); elle grandit mécaniquement à chaque module né (§8).
- **Explorer puis abandonner.** À chaque action testée, l’orchestrateur peut développer plusieurs sous-arbres et n’en **retenir qu’un** (la racine poussée en sortie). C’est le rôle de A^* : ouvrir les branches prometteuses, fermer les autres (budget borné, **a_etoile(budget_max=...)**).

6. Planification A^* à $g()/h()$ émergés

L’arbre : **nœuds** = **états prédits** s (au niveau d’abstraction choisi), **arêtes** = **actions** a (coût d’arête = effort/temps). $\text{voisins}(s) = [(\text{agir}(s,a), \text{coût}(a)) \text{ pour } a \quad]$. On réutilise **recherche.a_etoile** tel quel.

- **$g(s)$** = coût cumulé réel-ou-imaginé pour atteindre s (somme des $-r_t$ le long du chemin). C’est une **sortie de module** : le modèle de récompense/coût appris (Δfaim , douleur, temps). L’auteur : « un module qui va lui donner le $g()$ et $h()$ ».
- **$h(s)$** = estimation apprise du **reste à faire** jusqu’à l’objectif = **ValeurApprise** (**recherche.v_psi**), entraînée par **TD** (**entraîner_v_psi**) — d’abord 0 (A^* dégénère en Dijkstra borné, comportement sanctionné §7.3), puis informative à mesure que les nuits la façonnent (§7).
- **Niveau d’abstraction = levier de coût.** Le même A^* peut dérouler sur le champ brut (**predire_champ_sous_action**, cher, exact) ou sur un **état compact** (**agir** sur un latent, bon marché). On **encourage le haut du graphe** par le terme de coût déjà au cœur de l’orchestrateur : **valeur** = **progrès_objectif** - $\cdot$ **coût_calcul**. Prévoir en compact a un **coût_calcul** moindre $\rightarrow$ à progrès égal, l’orchestrateur préfère planifier haut. Si le compact devient non fiable (l’ancre au champ le révèle, **a_etoile_ancree**), il **redescend**.
- **Émergence du bon état.** L’état sur lequel planifier n’est pas donné : c’est le latent d’un module qui **réduit l’incertitude sur l’objectif**. Concrètement, le module qui rend $h()$ *ap-*

prenable (faible erreur TD) est celui dont le latent **sépare bien** les états « proche du but » des autres. On le sélectionne comme tout module : par la mesure (erreur TD de **h** = observable ; §28.4 peut régler le choix).

Jour comme nuit. Le jour, A^* tourne sous **budget serré** (temps réel) : peu de branches, on exploite **h**. La nuit, **budget large** : on déroule plus de possibilités, on **réarrange les modules** (essayer **agir** sur d'autres états), on met à jour **h** et **g** hors ligne — exactement « l'action nocturne explore plus avant les actions qu'on aurait pu faire, y compris arranger des modules différemment ».

7. Renforcement nocturne « de plus en plus amont »

C'est le cœur de l'apprentissage de navigation, et il **doit** être nocturne (crédit tardif). Récit et mécanisme :

1. **Jour** : l'agent, mû par la curiosité/faim, bouge ; par hasard il **percute un sucre**. L'épisode (graine : champ initial + actions + modules actifs + résidu) est capturé (mémoire épisodique, étape 12) — c'est un imprévu **récompensé**.
2. **Nuit** : on **rejoue** l'épisode et on fait un **backup TD à n pas** (ou $TD(n)$) sur **h** : la valeur de l'état **juste avant** le sucre monte (crédit à 1 pas). Au jeu suivant, l'état **2 pas avant** hérite d'une partie de cette valeur (), etc. Nuit après nuit, le crédit **remonte le fil** : $1 \rightarrow 2 \rightarrow \dots \rightarrow 6-15$ pas. `credit.rejeu_contrefactuel_nocturne` fournit la **baseline non biaisée** (ce qu'auraient donné les actions non prises), à poids figés.
3. **Résultat mesurable** : la **profondeur amont** à laquelle l'agent oriente correctement son action vers le sucre **croît avec le nombre de nuits**. C'est la courbe de validation de l'étape 20 (voir §10), l'analogue de la courbe $G(h)$ de l'étape 8.

Le **modèle agir** est ce qui rend ce jeu possible sans le monde : la nuit **imagine** les approches (dérouler **agir** depuis l'état initial de l'épisode vers le sucre) et évalue des variantes — impossible sans générateur de transition.

8. Exploiter tout ce qu'un nouveau module apporte

À chaque naissance, l'orchestrateur reçoit (et **doit** utiliser pour s'entraîner) :

- le **nouvel output** $\rightarrow$ un **élément pointable** de plus dans T_t (fenêtre dimensionnelle $\uparrow$) : `attention.construire_T_t` l'ajoute déjà ; il faut **amorcer** son embedding pour que l'attention le retrouve (`credit.amorçage_creation`, déjà écrit).
- le **générateur** $\rightarrow$ rend un futur **imaginable** sous ce module $\rightarrow$ nouvelle arête possible dans l'arbre A^* (**agir** peut s'appliquer à son latent).
- l'**état du condensateur** (statistiques du module) et la **qualité d'apprentissage** (incertitude, progrès) $\rightarrow$ **pondèrent la confiance** en rollout (`a_etoile_ancree` : confiance profondeur ; on la module aussi par la qualité du module) et **gate** son usage (un module immature n'est pas déroulé profond).
- le **contexte où il fonctionne** $\rightarrow$ `fiabilite_contextuelle` (déjà dans `attention`) : n'activer le module que là où il est fiable (activation creuse, §10.7).

Autrement dit : **une naissance enrichit à la fois l'état, l'arbre, l'heuristique et le budget** de l'orchestrateur. Ne pas recâbler tout cela serait gâcher l'information — c'est l'exigence explicite de l'auteur.

9. Alternatives considérées

- **(A) RL sans modèle** (REINFORCE direct sur les actions, à la Mode B actuel mais avec en sortie). *Rejeté comme socle* : ignore le modèle de transition émergé, ne planifie pas, et sur récompense rare converge très lentement. **Gardé comme composant** : la politique de Mode B fournit un **prior d'action** (quel a essayer en premier dans A*), ce qui guide la recherche — c'est le meilleur des deux mondes (plan + politique amortie).
 - **(B) Tout attention.py d'emblée** (Set Transformer + Pointer Net comme orchestrateur d'action complet). *Puissant mais différé* : difficile à tester incrémentalement. On **récolte** ses mécanismes (triplets, trace, REINFORCE, macro-pas) au fur et à mesure.
 - **(C) Retenu : opérateur agir typé + modèle de transition émergé + A* à g/h appris + déroulé continu + crédit nocturne amont**. Chaque pièce est **mesurable seule** et réutilise du code prouvé.
-

10. Plan d'implémentation (étapes 16→20, chacune mesurable)

Discipline inchangée : un harnais reproductible, des chiffres mesurés, commit + STATUS + tests verts à chaque palier ; **un entraînement GPU à la fois** (watchdog).

- **Étape 16 — Modèle de transition action-conditionné**. Module `agir(etat, action)→etat` (d'abord `etat = champ : predire_champ_sous_action`), né sur surprise confirmée, entraîné par accélération. *Mesure* : rappel de prédiction par action (comme étape 2a mais conditionné action) ; l'accél. nulle triviale, les autres apprises.
- **Étape 17 — Pulsions & objectif dominant**. Vecteur faim/douleur/curiosité/apprentissage/bullage/temps ; sélection dominante + réflexe (`decision_action`). *Mesure* : dynamique des pulsions sur un run, bascule d'objectif à l'hystérésis, réflexe qui court-circuite.
- **Étape 18 — Déroulé continu**. Boucle MPC : X pas imaginés, 1 action réelle, réinjection. L'orchestrateur émet actions+prédictions en flux (`macro_pas+trace`). *Mesure* : l'agent agit (immobile) quand la vision statique est maîtrisée ; sucre mangé **par hasard** non nul (base pour la nuit).
- **Étape 19 — Planification A***. `a_etoile` sur voisins = `agir·`, `g` = coût mesuré, `h` = ValeurApprise. Choix du **niveau d'abstraction** par `valeur = progrès - ·coût`. *Mesure* : à `h` fixée, la longueur/qualité du plan ; préférence effective pour l'état compact quand il suffit.
- **Étape 20 — Crédit nocturne amont**. Backup TD n-pas + replay contrefactuel sur les épisodes « sucre ». *Mesure* : **la profondeur amont d'orientation correcte croît avec le nombre de nuits** (1→...→6-15) — la courbe qui prouve la navigation apprise.

Jalon intermédiaire de démonstration : brancher ces objets sur le **viewer v7** (un panneau « pulsions » + l'arbre A* en cours + le plan retenu), pour *voir* l'agent délibérer.

11. Risques & garde-fous

- **Aucun câblage de tâche.** Zéro coordonnée d’objet, zéro distance-au-sucre dans la voie de décision : les pulsions sont des scalaires (faim = compteur interne, douleur = signal), la géométrie reste hors de la boucle (dette §27 tenue).
- **h non admissible.** `ValeurApprise` est best-first pondéré (Pohl), pas A^* admissible : acceptable (on veut vite-et-bon, pas l’optimum garanti). Le budget borné évite les dérives ; l’ancrage au champ rattrape les hallucinations d’état.
- **Explosion malgré tout.** Si l’arbre reste trop large, deux soupapes : réduire `exploré` au prior de Mode B (top-k actions), et remonter le niveau d’abstraction (coût $\uparrow$ décourage le brut). Les deux sont réglables par l’**auto-réglage §28.4** (étape 15) plutôt qu’à la main.
- **Crédit nocturne instable.** TD sur récompense rare peut diverger : clip de gradient déjà en place (`clip_grad_simulateur`), baseline contrefactuelle à poids figés (garde-fou **assert** dans `credit.py`), `< 1` pour borner la remontée.

Prochaine action : implémenter l’étape 16 (modèle de transition action-conditionné), premier maillon mesurable, en réutilisant `regime.py` (champ→champ) conditionné par l’action.